

PROYECTO FINAL

Sistema de Gestión para Salón de Belleza

Aplicación de escritorio desarrollada con C#, Windows Forms y SQL Server

Programación Orientada a Objetos

- **NOMBRES:** Víctor Eduardo Fernández Peña, Luis Angel Pimentel
- **PROFESOR:** José Antonio Jesús alcántara
- **MATRICULA:** 2024-0257, 2024-2573
- **MATERIA:** Programacion I

Introducción

El presente informe describe el desarrollo del Sistema de Gestión para Salón de Belleza. La aplicación fue creada para organizar las tareas que normalmente se registran de forma manual: clientes, empleados, servicios, citas y pagos. El proyecto utiliza una arquitectura por capas y aplica los principales conceptos de programación orientada a objetos en un caso práctico.

El sistema se encuentra preparado como una aplicación de escritorio en .NET 8. La interfaz se desarrolló con Windows Forms, el acceso a datos con ADO.NET y la persistencia con SQL Server. También se incluyeron procedimientos almacenados, validaciones, control de permisos y pruebas unitarias.

1. Idea del proyecto

La idea consiste en centralizar la administración diaria de un salón de belleza. En lugar de llevar la agenda, los cobros y los datos de los clientes en cuadernos o archivos separados, la aplicación permite trabajar con toda la información desde un mismo lugar.

Objetivo general

Desarrollar una aplicación de escritorio que permita gestionar las operaciones básicas del salón de forma ordenada, segura y con reglas que reduzcan errores al programar citas o registrar pagos.

Objetivos específicos

- Registrar, editar, consultar y desactivar clientes y empleados.
- Administrar servicios por categoría, precio y duración.
- Crear citas con uno o varios servicios y evitar conflictos de horario.
- Registrar pagos completos o parciales y calcular el saldo pendiente.
- Consultar el historial de atención y reportes básicos del negocio.
- Controlar el acceso según el rol de administrador o recepcionista.

Alcance

El proyecto incluye inicio de sesión, clientes, empleados, servicios, citas, pagos, historial y reportes. No contempla facturación fiscal, nómina, inventario, varias sucursales, reservas en línea ni una versión web.

2. Diseño preliminar

Desde el diseño inicial se separaron las responsabilidades en proyectos independientes. Esto evita que los formularios contengan toda la lógica y facilita localizar las entidades, las reglas y el acceso a la base de datos.

Capa	Proyecto	Responsabilidad
Presentación	SistemaSalon.WinForms	Formularios, navegación, captura de datos y mensajes al usuario.
Lógica	SistemaSalon.Logica	Validaciones, permisos, seguridad y reglas de citas y pagos.
Entidades	SistemaSalon.Entidades	Clases que representan clientes, empleados, servicios, citas y pagos.
Contratos	SistemaSalon.Interfaces	Interfaces para cálculo, agenda y repositorios genéricos.
Datos	SistemaSalon.Datos	Acceso a SQL Server mediante ADO.NET y procedimientos almacenados.
Pruebas	SistemaSalon.Tests	Pruebas unitarias de las principales reglas de negocio y conceptos de POO.

Relaciones principales entre clases

- Persona es la base abstracta de Cliente y Empleado.
- ServicioBase es la base de Servicio, ServicioCabello, ServicioUnas y ServicioFacial.
- Cita contiene una colección de DetalleCita y se relaciona con los pagos registrados.
- Cita implementa ICalculable e IAgendable.
- Los DAO de Cliente, Empleado y Servicio implementan IRepository<T>.

Evidencia en el proyecto: Documentation/DIAGRAMA_CLASES.md y los proyectos definidos en SistemaSalon.sln.

3. Implementación inicial

La implementación comenzó con las clases del dominio y las operaciones básicas necesarias para manipularlas. Se definieron objetos como Cliente, Empleado, Servicio, Cita, DetalleCita, Pago y Usuario. Después se agregaron los repositorios de datos, la lógica de negocio y finalmente los formularios de la aplicación.

Flujo principal implementado

1. El usuario inicia sesión con credenciales almacenadas de forma segura.

2. Se registran clientes, empleados y servicios.
3. A cada empleado se le asignan servicios y un horario laboral.
4. La cita se crea seleccionando cliente, empleado, fecha y servicios.
5. El sistema verifica disponibilidad y calcula duración y total.
6. Los pagos pueden registrarse de forma parcial hasta completar el saldo.
7. El historial y los reportes permiten revisar la información guardada.

Evidencia en el proyecto: formularios FrmLogin, FrmClientes, FrmEmpleados, FrmServicios, FrmCitas, FrmPagos, FrmHistorial y FrmReportes.

4. Clases normales

Las clases normales representan información concreta del sistema. Cada una contiene propiedades y métodos relacionados con su responsabilidad. No se utilizaron campos públicos; los datos se exponen mediante propiedades, lo que permite mantener control sobre los valores almacenados.

Clase	Información representada
Cliente	Datos personales, dirección, fecha de nacimiento, observaciones y fecha de registro.
Empleado	Datos personales, cédula, especialidad y fecha de contratación.
Cita	Cliente, empleado, fechas, estado, descuento, total y lista de servicios.
DetalleCita	Servicio seleccionado, cantidad, precio, duración y subtotal.
Pago	Monto, método de pago, referencia, fecha y estado del pago.
Usuario	Credenciales, rol, empleado asociado y estado de la cuenta.

Encapsulación

La clase Persona mantiene privados los campos de nombres, apellidos y correo. Las propiedades Nombres y Apellidos rechazan valores vacíos, mientras que Correo limpia espacios y permite un valor nulo. Esta validación protege el estado del objeto desde el momento en que se asignan los datos.

Evidencia en el proyecto: SistemaSalon.Entidades/Persona.cs.

5. Constructores

Se utilizaron constructores vacíos y constructores con parámetros. El constructor vacío facilita la creación de objetos desde los formularios y al convertir resultados de la base de datos. El constructor con parámetros permite inicializar de una vez los datos comunes heredados de Persona.

Ejemplo simplificado de la clase Cliente:

```
public Cliente(int id, string nombres, string apellidos, string telefono,
               string? correo, bool estado)
    : base(id, nombres, apellidos, telefono, correo, estado)
{
}
```

La llamada a base(...) evita repetir en Cliente y Empleado la inicialización de Id, nombres, apellidos, teléfono, correo y estado. De esta manera se aprovecha la herencia y se mantiene una única regla para los datos comunes.

Evidencia en el proyecto: SistemaSalon.Entidades/Persona.cs, Cliente.cs y Empleado.cs.

6. Clases abstractas

Persona

Persona reúne la información compartida por clientes y empleados. Al ser abstracta no se instancia directamente. Además, declara ObtenerDescripcion() como método abstracto, por lo que cada clase derivada debe proporcionar su propia descripción.

- Cliente devuelve una descripción con el nombre completo del cliente.
- Empleado incluye el nombre completo y su especialidad.

ServicioBase

ServicioBase contiene los datos comunes de cualquier servicio: nombre, descripción, precio, duración y estado. El método abstracto CalcularPrecio() permite que cada tipo de servicio aplique una regla diferente.

- ServicioCabello devuelve el precio base.
- ServicioUnas agrega 150 cuando incluye decoración.
- ServicioFacial agrega 200 cuando incluye mascarilla.

Este diseño también demuestra polimorfismo: una colección de ServicioBase puede contener diferentes tipos de servicio y ejecutar CalcularPrecio() sin conocer la clase concreta de cada objeto.

Evidencia en el proyecto: ServicioBase.cs, ServicioCabello.cs, ServicioUnas.cs, ServicioFacial.cs y la prueba Servicios_Aplican_Polimorfismo.

7. Sobrecargas

La sobrecarga se aplicó en la búsqueda de clientes. El nombre del método se mantiene, pero los parámetros cambian según el dato disponible. Esto evita crear nombres distintos para una misma intención de búsqueda.

```
BuscarCliente(clientes, int id)
BuscarCliente(clientes, string telefono)
BuscarCliente(clientes, string nombres, string apellidos)
```

Las mismas tres formas también están disponibles en ClienteDAO para consultar por identificador, teléfono o nombre completo. La prueba Sobrecarga_Busca_Cliente comprueba que las tres llamadas devuelven el cliente esperado.

Evidencia en el proyecto: SistemaSalon.Logica/CitaServicio.cs, SistemaSalon.Datos/ClienteDAO.cs y SistemaSalon.Tests/ReglasNegocioTests.cs.

8. Interfaces

Las interfaces se utilizaron como contratos. Indican qué operaciones debe ofrecer una clase sin imponer cómo deben implementarse.

Interfaz	Operaciones	Aplicación
ICalculable	CalcularTotal()	Cita calcula el subtotal de los detalles, aplica el descuento y devuelve el total.
IAgendable	VerificarDisponibilidad(), Confirmar(), Cancelar()	Cita valida su intervalo y servicios, además de cambiar su estado.
IRepositorio<T>	Insertar, Actualizar, Desactivar, ObtenerPorId y Listar	Los DAO mantienen una forma común de realizar operaciones de datos.

Evidencia en el proyecto: proyecto SistemaSalon.Interfaces, Cita.cs y los DAO que implementan IRepositorio<T>.

9. Optimización

La optimización no se limitó a reducir líneas de código. Se trabajó en la reutilización, el acceso eficiente a datos y la prevención de operaciones innecesarias o inconsistentes.

Acceso a datos

- DaoBase concentra la creación de comandos, parámetros, adaptadores y tablas de resultados.
- Las conexiones y comandos usan using, por lo que los recursos se liberan al finalizar cada operación.
- Los procedimientos almacenados y SqlParameter reducen consultas repetidas y evitan concatenar entradas del usuario.
- Los índices de SQL Server aceleran búsquedas por nombres, fechas, cliente, empleado, estado y pagos.

Reglas y memoria

- Las colecciones se consultan con Sum, GroupBy y FirstOrDefault, evitando recorridos manuales más extensos.
- Las validaciones detienen operaciones inválidas antes de guardar información en la base de datos.
- La separación por capas permite reutilizar la lógica sin duplicarla en cada formulario.
- Las consultas de agenda verifican solapamientos mediante comparación de intervalos de fecha y hora.

Evidencia en el proyecto: DaoBase.cs, CitaServicio.cs, procedimientos almacenados e índices definidos en Database/03_CreateIndexes.sql.

10. Presentación y funcionamiento

Para presentar el sistema conviene seguir un caso completo, porque permite demostrar en pocos minutos la interfaz, las reglas del negocio, la base de datos y los conceptos de POO.

8. Iniciar sesión como administrador y mostrar el menú principal.
9. Registrar un cliente y un empleado.
10. Crear un servicio y asignarlo al empleado.
11. Definir el horario laboral del empleado.
12. Crear una cita con uno o varios servicios.
13. Confirmar la cita y cambiarla a En proceso.
14. Registrar un pago parcial y luego el pago restante.
15. Completar la cita y consultar el historial y los reportes.

Validaciones que se pueden demostrar

- No se permiten citas en el pasado ni fuera del horario del empleado.
- No se permiten citas solapadas para el empleado o el cliente.
- Una cita confirmada debe pasar por En proceso antes de completarse.
- No se puede cancelar una cita que tenga pagos activos.
- Un pago no puede ser mayor que el saldo pendiente.
- El recepcionista no puede entrar a módulos reservados para el administrador.

Pruebas realizadas

El proyecto incluye pruebas unitarias para nombre completo, total y duración de citas, saldo pendiente, pagos, cancelaciones, servicios duplicados, horarios, permisos, transiciones de estado, sobrecarga, polimorfismo y validaciones comunes. Estas pruebas sirven como respaldo de que la lógica principal responde de la forma esperada.

Conclusiones

El proyecto cumple con el objetivo de administrar las operaciones esenciales de un salón de belleza mediante una aplicación de escritorio conectada a SQL Server. La organización por capas permitió separar la interfaz, las reglas de negocio, las entidades y el acceso a datos.

También se aplicaron de forma directa los conceptos solicitados en la asignatura: clases normales, constructores, encapsulación, herencia, clases abstractas, métodos abstractos, polimorfismo, sobrecarga e interfaces. Las validaciones, los índices y las pruebas unitarias completan el trabajo y ayudan a que el sistema sea más estable y fácil de mantener.

Archivos principales consultados

- README.md y Documentation/DESCRIPCION_PROYECTO.md.
- Documentation/DIAGRAMA_CLASES.md y Documentation/POO_APLICADA.md.
- SistemaSalon.Entidades, SistemaSalon.Interfaces, SistemaSalon.Logica y SistemaSalon.Datos.
- Database/00_BaseDatosCompleta.sql y Database/03_CreateIndexes.sql.
- SistemaSalon.Tests/ReglasNegocioTests.cs.